

opencode-project-context

Samouczek użytkownika

Wersja 1.0 · Lipiec 2026 · Plugin do OpenCode

Spis treści

1. Co to jest i jakie daje korzyści
2. Szybka instalacja
3. Szybki start — minimalny zestaw komend
4. Wszystkie polecenia
5. Konfiguracja pluginu
6. Przykładowy use case

1. Co to jest i jakie daje korzyści

opencode-project-context to lokalny plugin do OpenCode, który ogranicza zużycie tokenów i zapamiętuje informacje istotne dla rozwoju projektu. Działa lokalnie, offline, bez zewnętrznych API i bez dodatkowego modelu LLM.

Korzyści:

- **Redukcja tokenów wejściowych** — skracanie długich wyników narzędzi (testy, build, `git diff`, `grep`) z zachowaniem kodu wyjścia, pierwszego błędu i podsumowania.
- **Trwała pamięć projektu per repozytorium** — fakty architektoniczne, konwencje i komendy w `project-facts.md`; handoff sesji w `active-session.json`.
- **Deduplikacja odczytów** — drugi identyczny odczyt tego samego pliku nie jest ponownie wysyłany do modelu (trwały cache LRU na dysku).
- **Auto-ekstrakcja faktów** — plugin zbiera statystyki komend, edytowanych plików i blokerów, a Ty zatwierdzasz propozycje do `project-facts.md`.
- **Pamięć stanu testów w czasie** — historia uruchomień z kodem wyjścia, listą failed i git SHA, bez ponownej analizy logów.
- **Wykrywanie i cofanie regresji** — korelacja padającego testu ze zmianami w repo i bezpieczny powrót do ostatniej działającej wersji.
- **Bezpieczeństwo** — blokada odczytu `.env` /kluczy, maskowanie sekretów w wynikach i artefaktach.

2. Szybka instalacja

Najszybsza metoda to skrypt `install.sh`. Wymaga shella kompatybilnego z POSIX (Git Bash na Windows, bash/zsh na Linuksie/macOS). Skrypt jest **idempotentny** — można uruchamiać wielokrotnie bez nadpisywania istniejącej konfiguracji i faktów.

W nowym projekcie

```
mkdir ~/moj-projekt && cd ~/moj-projekt
git init
# skopiuj katalog pluginu do projektu, następnie:
bash install.sh
```

W istniejącym projekcie

```
bash install.sh /sciezka/do/istniejacego-repo
```

Skrypt automatycznie:

- Kopiuje `project-context.ts` do `.opencode/plugins/`
- Kopiuje komendy `memory.md`, `context.md`, `regression.md` do `.opencode/command/`
- Tworzy katalog `.opencode/memory/` i szablon `project-facts.md` (jeśli brak)
- Tworzy `opencode.json` z rejestracją pluginu i konfiguracją (jeśli brak)
- Dopisuje wykluczenia do `.gitignore`

Po instalacji **zrestartuj OpenCode** — konfiguracja nie przeładowuje się na gorąco.

3. Szybki start — minimalny zestaw komend

Po restarcie OpenCode wpisz w TUI następujące komendy, aby plugin zaczął działać:

```
# 1. Uzupełnij fakty projektu (jednorazowo)
#   Edytuj .opencode/memory/project-facts.md:
#       # Architektura
#       - Firmware: ESP-IDF, komponenty w components/
#       # Komendy
#       - Build: idf.py build
#       - Testy: pytest -q tests/

# 2. Zweryfikuj, że plugin działa
/memory status

# 3. Wykonaj zwykłą pracę (uruchamiaj testy, edytuj pliki)
#   Plugin automatycznie skraca wyniki i zbiera statystyki

# 4. Po pracy – zachowaj handoff
/memory save

# 5. Po kilku sesjach – zobacz propozycje faktów
/memory propose
/memory commit
```

To wystarczy, by plugin zaczął ograniczać tokeny i budować pamięć projektu. Reszta dzieje się automatycznie w tle.

4. Wszystkie polecenia

/memory <subkomenda> — zarządzanie pamięcią

Subkomenda	Działanie
status	Przegląd stanu: worktree, rozmiar faktów, sesja, cache, artefakty, metryki
show	Pełna zawartość: <code>project-facts.md</code> , <code>active-session.json</code> , wstrzyknięty kontekst, historia testów, trace
save	Wymusza zapis bieżącego handoffu do <code>active-session.json</code>
clear-session	Czyści cache i stan sesji (zachowuje <code>project-facts.md</code> i zagregowany trace)
clear-project	Usuwa całą pamięć projektu (w tym <code>project-facts.md</code>)
compact	Ręcznie tworzy krótki handoff
propose	Generuje propozycje faktów z sesji (komendy, hotspoty, nieudane testy, blokery)
commit	Dopisuje propozycje do <code>project-facts.md</code> pod markerem HTML
test-history	Historia uruchomień testów/buildów (najnowsze na górze)

/context <subkomenda> — diagnostyka budżetu

Subkomenda	Działanie
budget	Wykorzystanie limitów tokenów/linii vs skonfigurowane maksima
artifacts	Lista ostatnich artefaktów (pełnych wyników) do selektywnego odczytu

/regression <subkomenda> — wykrywanie i cofanie regresji

Subkomenda	Działanie
last-good	Pokazuje okno regresji: ostatni zielony i pierwszy czerwony uruchomienie + git SHA
suspect	Podejrzane pliki i commity zmienione między last-good a first-red, posortowane wg prawdopodobieństwa
revert <plik>	Przywraca plik do wersji last-good (wymaga confirm w trybie bezpiecznym)
revert confirm <plik>	Wykonuje git checkout <sha> -- <plik>
revert all	Przywraca wszystkie podejrzane pliki (wymaga confirm all)
revert all confirm	Wykonuje przywrócenie wszystkich podejrzanych plików
revert stash	git stash wszystkich niezatwierdzonych zmian (zawsze bezpieczne, odwracalne)

Bezpieczeństwo: Plugin nigdy nie wykonuje git reset --hard ani git clean . stash jest odwracalny (git stash pop). checkout <sha> -- <plik> nadpisuje tylko wskazane pliki — cofnięcie przez git checkout HEAD -- <plik> .

Narzędzie agenta: read_artifact

Plugin rejestruje narzędzie read_artifact dostępne dla agenta. Służy do pobierania pełnych wyników zapisanych jako artefakty, fragmentami:

```
read_artifact({ artifactId: "a71d8c", offset: 0, limit: 100, search: "error" })
```

Parametry: artifactId (krótki hash), offset / limit (paginacja), search (filtrowanie linii). Dzięki temu agent pobiera tylko interesujący fragment logu zamiast pełnego outputu.

5. Konfiguracja pluginu

Wszystkie opcje w sekcji contextOptimizer w opencode.json są opcjonalne — brak sekcji oznacza wartości domyślne.

Pole	Domyślnie	Opis
<code>enabled</code>	<code>true</code>	Globalny wyłącznik pluginu
<code>maxProjectMemoryTokens</code>	<code>1500</code>	Limit wstrzykiwanego kontekstu na start sesji
<code>maxSessionHandoffTokens</code>	<code>1000</code>	Limit serializowanego handoffu
<code>maxToolResultLines</code>	<code>100</code>	Globalny limit linii wyniku narzędzia
<code>maxDiffLines</code>	<code>120</code>	Limit linii skróconego <code>git diff</code>
<code>maxSearchMatches</code>	<code>40</code>	Limit dopasowań <code>grep</code> / <code>rg</code>
<code>maxArtifactPreviewLines</code>	<code>80</code>	Domyślny limit podglądu <code>read_artifact</code>
<code>deduplicateReadResults</code>	<code>true</code>	Deduplikacja powtórzonych odczytów
<code>storeFullArtifacts</code>	<code>true</code>	Zapis pełnych wyników jako artefakty
<code>persistentDedupCache</code>	<code>true</code>	Zapis dedup-cache na dysk (zachowanie po restarcie)
<code>maxDedupCacheEntries</code>	<code>500</code>	Maksymalna liczba wpisów w LRU
<code>maxTestHistoryEntries</code>	<code>50</code>	Maksymalna liczba zapamiętanych uruchomień testów
<code>regressionTrackHead</code>	<code>true</code>	Zapisuj git SHA przy każdym uruchomieniu testów
<code>regressionSafeRevertOnly</code>	<code>true</code>	Wymagaj <code>confirm</code> przy checkout; bez auto <code>--hard</code>

Przykład pełnej konfiguracji:

```
{
  "$schema": "https://opencode.ai/config.json",
  "plugin": ["/.opencode/plugins/project-context.ts"],
  "contextOptimizer": {
    "enabled": true,
    "maxProjectMemoryTokens": 1500,
    "maxSessionHandoffTokens": 1000,
    "maxToolResultLines": 100,
    "maxDiffLines": 120,
    "maxSearchMatches": 40,
    "maxArtifactPreviewLines": 80,
    "deduplicateReadResults": true,
    "storeFullArtifacts": true,
    "persistentDedupCache": true,
    "maxDedupCacheEntries": 500,
    "maxTestHistoryEntries": 50,
    "regressionTrackHead": true,
    "regressionSafeRevertOnly": true
  }
}
```

Zmiany konfiguracji wymagają restartu OpenCode.

6. Przykładowy use case

Scenariusz: projekt embedded ESP-IDF. Agent naprawia retry transmisji radiowej. Po zmianach testy przestają przechodzić.

Krok 1 — Start sesji

Plugin przy `session.created` wstrzykuje związły kontekst:

```
PROJECT MEMORY
Architecture: ESP-IDF; FreeRTOS queues; no blocking in ISR.
Rules: no dynamic allocation in critical paths; public API requires tests.
Previous task: Retry after downlink timeout.
Changed files: src/radio/retry.c, tests/test_retry.py.
Last test: 1 failed, 14 passed.
```

Krok 2 — Praca agenta

Agent czyta `src/radio/retry.c`, edytuje logikę backoff, uruchamia testy:

```
pytest -q tests/test_retry.py
```

Plugin skraca wynik (oryginalnie 450 linii → 12 linii):

```
Command: pytest -q tests/test_retry.py (exit 1)
Summary: 1 failed, 14 passed.
Failure: tests/test_retry.py:87
Expected retry delay: 4 s; actual: 2 s.
Relevant source: src/radio/retry.c:114.
Full output available: artifact://a71d8c
```

Plugin zapisuje `TestRun` do `test-history.json` z git SHA, aktualizuje `session-trace.json` (edytowane pliki, komenda).

Krok 3 — Wykrycie regresji

W kolejnej sesji agent uruchamia testy — `test_retry_delay` znów pada. Użytkownik diagnozuje:

```
/regression last-good
```

Wynik:

```
=== Regression window ===
Last good: 2026-07-30T14:55:01Z exit=0 idf.py build
          HEAD: a1b2c3d
First red: 2026-07-30T15:12:33Z exit=1 pytest
          HEAD: e4f5a6b
Failing test: tests/test_retry.py::test_retry_delay
```

```
/regression suspect
```

Wynik:

```
=== Regression suspects ===
Failing test: tests/test_retry.py::test_retry_delay
Okno: 14:55 → 15:12
Committed w oknie:
  c3d4e5f fix: retry backoff
Pliki zmienione w oknie:
  src/radio/retry.c
  src/radio/backoff.c
```

Krok 4 — Cofnięcie regresji

Użytkownik przywraca podejrzany plik do ostatniej działającej wersji:

```
/regression revert src/radio/retry.c
# → prosi o confirm (tryb bezpieczny)
/regression revert confirm src/radio/retry.c
# → Przywrócono src/radio/retry.c do wersji a1b2c3d.
```

Agent uruchamia testy ponownie — przechodzą. Winny wskazany: zmiana w `retry.c`. Jeśli użytkownik chce wrócić do zmian roboczych:

```
/regression revert stash
```

Krok 5 — Zapis wiedzy

Po kilku sesjach plugin zebrał statystyki. Użytkownik generuje propozycje faktów:

```
/memory propose
```

Wynik:

```
## Komendy
- pytest (uruchamiano 8x)
- idf.py build (uruchamiano 3x)

## Hotspoty
- src/radio/retry.c (12x edytowany)
- tests/test_retry.py (7x)

## Ostatnie nieudane testy
- [2026-07-30T15:12Z] pytest (exit 1): test_retry_delay

## Ryzyka
- test_retry_delay (powtarzający się błąd)
```

```
/memory commit
```

Propozycje dopisane do `project-facts.md`. Od teraz każda nowa sesja otrzymuje tę wiedzę automatycznie, a agent unika powtarzania tych samych błędów.

Więcej szczegółów: `README.md` w repozytorium pluginu. Plugin działa w trybie fail-open — błędy nie zatrzymują OpenCode, diagnostyka w `.opencode/memory/plugin-errors.log`.